JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class AbstractSequentialList<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractList<E>
      java.util.AbstractSequentialList<E>

All Implemented Interfaces:

`Iterable<E>, Collection<E>, List<E>`

Direct Known Subclasses:

`LinkedList`

```
public abstract class AbstractSequentialList<E>
extends AbstractList<E>
```

This class provides a skeletal implementation of the `List` interface to minimize the effort required to implement this interface backed by a "sequential access" data store (such as a linked list). For random access data (such as an array), `AbstractList` should be used in preference to this class.

This class is the opposite of the `AbstractList` class in the sense that it implements the "random access" methods (`get(int index)`, `set(int index, E element)`, `add(int index, E element)` and `remove(int index)`) on top of the list's list iterator, instead of the other way around.

To implement a list the programmer needs only to extend this class and provide implementations for the `listIterator` and `size` methods. For an unmodifiable list, the programmer need only implement the list iterator's `hasNext`, `next`, `hasPrevious`, `previous` and `index` methods.

For a modifiable list the programmer should additionally implement the list iterator's `set` method. For a variable-size list the programmer should additionally implement the list iterator's `remove` and `add` methods.

The programmer should generally provide a void (no argument) and collection constructor, as per the recommendation in the `Collection` interface specification.

This class is a member of the Java Collections Framework.

Since:

1.2

See Also:

`Collection`, `List`, `AbstractList`, `AbstractCollection`

Field Summary

Fields inherited from class java.util.AbstractList

`modCount`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**`AbstractSequentialList`**()	Sole constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
void	**`add`**(int index, **`E`** element)	Inserts the specified element at the specified position in this list (optional operation).
boolean	**`addAll`**(int index, **`Collection`**<? extends **`E`**> c)	Inserts all of the elements in the specified collection into this list at the specified position

		(optional operation).
E	**get**(int index)	Returns the element at the specified position in this list.
Iterator<E>	**iterator**()	Returns an iterator over the elements in this list (in proper sequence).
abstract **ListIterator<E>**	**listIterator**(int index)	Returns a list iterator over the elements in this list (in proper sequence).
E	**remove**(int index)	Removes the element at the specified position in this list (optional operation).
E	**set**(int index, **E** element)	Replaces the element at the specified position in this list with the specified element (optional operation).

Methods inherited from class java.util.AbstractList

add, clear, equals, hashCode, indexOf, lastIndexOf, listIterator, removeRange, subList

Methods inherited from class java.util.AbstractCollection

addAll, contains, containsAll, isEmpty, remove, removeAll, retainAll, size, toArray, toArray, toString

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.List

addAll, contains, containsAll, isEmpty, remove, removeAll, replaceAll, retainAll, size, sort, spliterator, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

AbstractSequentialList

```
protected AbstractSequentialList()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

get

```
public E get(int index)
```

Returns the element at the specified position in this list.

This implementation first gets a list iterator pointing to the indexed element (with `listIterator(index)`). Then, it gets the element using `ListIterator.next` and returns it.

Specified by:

`get` in interface `List<E>`

Specified by:

`get` in class `AbstractList<E>`

Parameters:

`index - index of the element to return`

Returns:

`the element at the specified position in this list`

Throws:

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index >= size())

set

```
public E set(int index,
             E element)
```

Replaces the element at the specified position in this list with the specified element (optional operation).

This implementation first gets a list iterator pointing to the indexed element (with `listIterator(index)`). Then, it gets the current element using `ListIterator.next` and replaces it with `ListIterator.set`.

Note that this implementation will throw an `UnsupportedOperationException` if the list iterator does not implement the `set` operation.

Specified by:

`set` in interface `List<E>`

Overrides:

`set` in class `AbstractList<E>`

Parameters:

`index` - index of the element to replace

`element` - element to be stored at the specified position

Returns:

the element previously at the specified position

Throws:

`UnsupportedOperationException` - if the set operation is not supported by this list

`ClassCastException` - if the class of the specified element prevents it from being added to this list

`NullPointerException` - if the specified element is null and this list does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index >= size())

add

```
public void add(int index,
                E element)
```

Inserts the specified element at the specified position in this list (optional operation). Shifts the element currently at that position (if any) and any subsequent elements to the right (adds one to their indices).

This implementation first gets a list iterator pointing to the indexed element (with `listIterator(index)`). Then, it inserts the specified element with `ListIterator.add`.

Note that this implementation will throw an `UnsupportedOperationException` if the list iterator does not implement the `add` operation.

Specified by:

`add` in interface `List<E>`

Overrides:

`add` in class `AbstractList<E>`

Parameters:

`index` - index at which the specified element is to be inserted

`element` - element to be inserted

Throws:

`UnsupportedOperationException` - if the add operation is not supported by this list

`ClassCastException` - if the class of the specified element prevents it from being added to this list

`NullPointerException` - if the specified element is null and this list does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this list

`IndexOutOfBoundsException` - if the index is out of range (index < 0 || index > size())

remove

```
public E remove(int index)
```

Removes the element at the specified position in this list (optional operation). Shifts any subsequent elements to the left (subtracts one from their indices). Returns the element that was removed from the list.

This implementation first gets a list iterator pointing to the indexed element (with `listIterator(index)`). Then, it removes the element with `ListIterator.remove`.

Note that this implementation will throw an `UnsupportedOperationException` if the list iterator does not implement the `remove` operation.

Specified by:

`remove` in interface `List<E>`

Overrides:

`remove` in class `AbstractList<E>`

Parameters:

`index - the index of the element to be removed`

Returns:

`the element previously at the specified position`

Throws:

`UnsupportedOperationException - if the remove operation is not supported by this list`

`IndexOutOfBoundsException - if the index is out of range (index < 0 || index >= size())`

addAll

```
public boolean addAll(int index,
                      Collection<? extends E> c)
```

Inserts all of the elements in the specified collection into this list at the specified position (optional operation). Shifts the element currently at that position (if any) and any subsequent elements to the right (increases their indices). The new elements will appear in this list in the order that they are returned by the specified collection's iterator. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress. (Note that this will occur if the specified collection is this list, and it's nonempty.)

This implementation gets an iterator over the specified collection and a list iterator over this list pointing to the indexed element (with `listIterator(index)`). Then, it iterates over the specified collection, inserting the elements obtained from the iterator into this list, one at a time, using `ListIterator.add` followed by `ListIterator.next` (to skip over the added element).

Note that this implementation will throw an `UnsupportedOperationException` if the list iterator returned by the `listIterator` method does not implement the `add` operation.

Specified by:

addAll in interface List<E>

Overrides:

addAll in class AbstractList<E>

Parameters:

index - index at which to insert the first element from the specified collection

c - collection containing elements to be added to this list

Returns:

true if this list changed as a result of the call

Throws:

UnsupportedOperationException - if the addAll operation is not supported by this list

ClassCastException - if the class of an element of the specified collection prevents it from being added to this list

NullPointerException - if the specified collection contains one or more null elements and this list does not permit null elements, or if the specified collection is null

IllegalArgumentException - if some property of an element of the specified collection prevents it from being added to this list

IndexOutOfBoundsException - if the index is out of range (index < 0 || index > size())

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this list (in proper sequence).

This implementation merely returns a list iterator over the list.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `List<E>`

Overrides:

`iterator` in class `AbstractList<E>`

Returns:

an iterator over the elements in this list (in proper sequence)

listIterator

```
public abstract ListIterator<E> listIterator(int index)
```

Returns a list iterator over the elements in this list (in proper sequence).

Specified by:

`listIterator` in interface `List<E>`

Overrides:

`listIterator` in class `AbstractList<E>`

Parameters:

index - index of first element to be returned from the list iterator (by a call to the next method)

Returns:

a list iterator over the elements in this list (in proper sequence)

Throws:

C

```
IndexOutOfBoundsException - if the index is out of range (index < 0 || index > size())
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.